Pneumonia Detection Using Convolutional Neural Network (CNN)

📌 Project Overview

This project uses a custom Convolutional Neural Network (CNN) to classify chest X-ray images into two categories: **Pneumonia** and **Normal**. The model is developed using TensorFlow and Keras in Google Colab.

The project applies data augmentation, Batch Normalization, and Dropout to improve model generalization and reduce overfitting. Model performance is evaluated using accuracy, loss curves, a confusion matrix, precision, recall, F1-score, and ROC-AUC.

🎯 Objectives

- Classify chest X-ray images as Normal or Pneumonia.
- Design a custom CNN architecture.
- Apply data augmentation to improve generalization.
- Use Batch Normalization and Dropout during training.
- Visualize training and validation accuracy and loss.
- Evaluate the model using suitable classification metrics.

🛠️ Technologies Used

- Python
- Google Colab
- TensorFlow and Keras
- NumPy
- Matplotlib
- Scikit-learn
- Kaggle API

📁 Dataset

Dataset: Chest X-Ray Images (Pneumonia)

Source: Kaggle Dataset

The dataset contains chest X-ray images organized into:

- train — training images
- val — validation images

- test — testing images

Each split contains two classes:

- NORMAL
- PNEUMONIA

🧠 Model Architecture

The custom CNN includes:

1. Convolutional layers for feature extraction.
2. Batch Normalization for training stability.
3. Max Pooling for spatial downsampling.
4. Dropout for reducing overfitting.
5. Global Average Pooling.
6. A Dense layer for classification.
7. A Sigmoid output layer for binary classification.

Data Preprocessing

- Image resizing to 150 × 150 pixels.
- Pixel normalization to the range 0–1.
- Random rotation, zoom, and translation for data augmentation.

⚙️ Project Workflow

1. Configure Kaggle API credentials in Google Colab.
2. Download and extract the dataset.
3. Load and preprocess chest X-ray images.
4. Build the custom CNN model.
5. Train the model using training and validation data.
6. Plot accuracy and loss curves.
7. Evaluate the model on the test dataset.
8. Generate a confusion matrix and classification report.
9. Save the trained model in Keras format.

📊 Evaluation Metrics

The model is evaluated using:

- **Accuracy:** Overall proportion of correct predictions.
- **Precision:** Proportion of predicted positive cases that are correct.
- **Recall:** Proportion of actual positive cases correctly identified.
- **F1-score:** Harmonic mean of precision and recall.
- **ROC-AUC:** Ability to distinguish between the two classes.
- **Confusion Matrix:** Summary of correct and incorrect predictions.

Training and validation accuracy/loss graphs are used to assess model learning and potential overfitting.

🚀 How to Run the Project

1. Open Google Colab.
2. Upload or open the project notebook.
3. Configure your Kaggle API credentials securely.
4. Run the notebook cells in order to download the dataset, preprocess images, and train the model.
5. Review the evaluation metrics and visualizations.
6. Save the trained model as pneumonia_cnn_final.keras.

📁 Suggested Repository Structure

Pneumonia-Detection-CNN/

├── Pneumonia_Detection_CNN.ipynb

├── README.md

├── accuracy_loss.png

├── confusion_matrix.png

└── pneumonia_cnn_final.keras

The model file and generated images are optional repository contents. Consider using Git LFS or another suitable storage method for large model files. Never upload Kaggle credentials.

🔮 Future Improvements

- Experiment with different CNN architectures and hyperparameters.

- Evaluate class imbalance and use suitable balancing techniques.
- Compare the custom CNN with pretrained models such as DenseNet or EfficientNet.
- Add an image-upload interface using FastAPI.
- Evaluate performance on an independent external dataset.

⚠️ Disclaimer

This project is intended for educational and research purposes only. It is not a clinically validated diagnostic tool and must not be used to diagnose or make medical decisions.

👨‍💻 Author

Developed as part of an AI/ML assignment focused on deep learning and medical image classification.